



5. Vectores y Matrices en Python y C.

Profesor: Juan Moreno García

Índice

1. Objetivos.
2. Listas y tuplas en Python.
3. Vectores en C.
4. Matrices en C.
5. Ejemplos.
6. Ejercicios.
7. Conclusiones.

1. Objetivos

1. Recordar las **listas y tuplas de Python (if)**.
2. Aprender los **vectores de C y ver las diferencias con listas y tuplas**.
3. Aprender las **matrices de C y ver las diferencias con listas y tuplas**.
4. Saber crear funciones que **utilicen vectores y matrices como parámetros para introducir información** en la misma y **devolver información** mediante un vector.

2. Listas y tuplas en Python.

- * Python tiene **dos potentes estructuras** que son las **listas y tuplas**.
- * Ambas pueden **contener elementos de distintos tipos**, simples y compuestos, **encargándose Python de la gestión de memoria** necesaria durante el trabajo en los programas Python.
- * C no tiene nada parecido. **Lo más “parecido” pueden ser los vectores**. Aunque el parecido es escaso o incluso nulo.

3. Vectores en C.

Declaración de vectores

```
int nombreVector[tamaño];
```

```
int v[10]; // Se accede con los índices del 0 al 9  
float f[6]; // Se accede con los índices del 0 al 5
```

Ejemplo de uso de vectores

```
void rellenaVectorEnteroAleatorioEntre(int v[], int tam, int a, int b) {  
    int i;  
    // Inicializa el vector v con valores aleatorios  
    for(i=0; i<tam; i++) {  
        v[i] = random(a, b);  
    }  
}
```

```
// Funcion que muestra un vector float de tamaño tam  
void muestraVectorReal(float f[], int tam) {  
    int i;  
    for(i=0; i<tam;i++) {  
        if (i!=tam-1) {  
            Serial.print( String(f[i]) + ", " );  
        } else {  
            Serial.println( String(f[i]) );  
        }  
    }  
}
```

```
// Funcion que muestra un vector float de tamaño tam  
void muestraVectorEntero(int v[], int tam) {  
    int i;  
    for(i=0; i<tam;i++) {  
        if (i!=tam-1) {  
            Serial.print( String(v[i]) + ", " );  
        } else {  
            Serial.println( String(v[i]) );  
        }  
    }  
}
```

3. Vectores en C.

Programa que inicializa el vector entero v aleatoriamente, después rellena el vector float f con el valor resultado de la expresión $f[i] = v[i] + v[i]*0.5$.

```
# define TAM 10

void rellenaConFuncion(float f[], int v[], int tam) {
    int i;
    // Inicializa f[i] con el valor de v[i] más la mitad de v[i]
    for(i=0; i<tam;i++) {
        f[i] = v[i] + v[i]*0.5;
    }
}

void setup() {
    Serial.begin(115200);
    randomSeed(analogRead(0));
}

void loop() {
    int i, v[TAM]; // Se accede con los índices del 0 al TAM-1
    float f[TAM]; // Se accede con los índices del 0 al TAM-1

    // Inicializa el vector v con valores aleatorios
    rellenaVectorEnteroAleatorio(v, TAM);
    // Inicializa f[i] con el valor de v[i] más la mitad de v[i]
    rellenaConFuncion(f, v, TAM);
    // Muestra el vector v
    muestraVectorEntero(v, TAM);
    // Muestra el vector f
    muestraVectorReal(f, TAM);

    delay(3000);
}
```

4. Matrices en C.

Declaración de matrices

```
int m[5][8];
```

Ejemplo de uso de matrices

```
void rellenaMatrizAleatoria(int m[FIL][COL], int n_fil, int n_col) {  
    int i, j;  
    for(i=0;i<n_fil;i++) {  
        for(j=0;j<n_col;j++) {  
            m[i][j] = random(0,10);  
        }  
    }  
}
```

```
void muestraMatriz(int m[FIL][COL], int n_fil, int n_col){  
    int i, j;  
    String s;  
  
    Serial.println("-----");  
    for(i=0;i<n_fil;i++) {  
        s = String("");  
        for(j=0;j<n_col;j++) {  
            if(j==0) s += String("[") +String(m[i][j]);  
            else s += String(", ") + String(m[i][j]);  
        }  
        s += String("]");  
        Serial.println(s);  
    }  
}
```

5. Ejemplos.

Programa que primero rellena vector con valores aleatorios entre dos números solicitados por el Monitor Serie, después calcula el máximo mediante la función *maximoDeVectorEntero()*, finalmente, muestra por pantalla el vector y el valor máximo.

```
int maximoDeVectorEntero(int v[], int tam) {
    int i, maximo;
    maximo = v[0];
    for(i=0; i<tam; i++) {
        if (v[i]>maximo) {
            maximo = v[i];
        }
    }
    return maximo;
}

void rellenaVectorEnteroAleatorioEntre(int v[], int tam, int a, int b) {
    int i;
    // Inicializa el vector v con valores aleatorios
    for(i=0; i<tam; i++) {
        v[i] = random(a, b);
    }
}
```


5. Ejemplos.

```
# define TAM 10

void setup() {
    Serial.begin(115200);
    randomSeed(analogRead(0));
}

void loop() {
    int a, b, maximo, v[TAM]; // Se accede con los índices del 0 al TAM-1

    // Lee un valor entero del Monitor Serie
    a = leerEntero("Dame un valor entero");
    // Lee un valor entero del Monitor Serie
    b = leerEntero("Dame otro valor entero");
    // Inicializa el vector v con valores aleatorios
    rellenaVectorEnteroAleatorioEntre(v, TAM, a, b);
    // Devuelve el máximo del vector en maximo
    maximo = maximoDeVectorEntero(v, TAM);
    // Muestra el vector v
    muestraVectorEntero(v, TAM);
    // Muestra el máximo
    Serial.println( "El maximo del vector es " + String(maximo) );
}
```

5. Ejemplos

Programa que primero rellena una matriz con valores aleatorios entre dos números solicitados por el *Monitor Serie*, después calcula el máximo mediante la función ***maximoDeMatrizEnteros()***, finalmente, muestra por pantalla la matriz y el valor máximo obtenido.

```
int maximoDeMatrizEntero(int m[FIL][COL], int n_fil, int n_col) {
    int i, j, maximo;
    maximo = m[0][0];
    for(i=0; i<n_fil; i++) {
        for(j=0; j<n_col; j++) {
            if (m[i][j]>maximo) {
                maximo = m[i][j];
            }
        }
    }
    return maximo;
}

void rellenaMatrizEnteroAleatorioEntre(int m[FIL][COL], int n_fil, int n_col, int a, int b) {
    int i, j;
    // Inicializa la matriz m con valores aleatorios
    for(i=0; i<n_fil; i++) {
        for(j=0; j<n_col; j++) {
            m[i][j] = random(a, b);
        }
    }
}
```

5. Ejemplos

```
#define FIL 3
#define COL 4

void setup() {
    Serial.begin(115200);
    randomSeed(analogRead(0));
}

void loop() {
    int a, b, maximo, m[FIL][COL];

    // Lee un valor entero del Monitor Serie
    a = leerEntero("Dame un valor entero");
    // Lee un valor entero del Monitor Serie
    b = leerEntero("Dame otro valor entero");
    // Inicializa la matriz m con valores aleatorios
    rellenaMatrizEnteroAleatorioEntre(m, FIL, COL, a, b);
    // Devuelve el maximo de la matriz y lo copia en maximo
    maximo = maximoDeMatrizEntero(m, FIL, COL);
    // Muestra la matriz m
    muestraMatrizEntera(m, FIL, COL);
    // Muestra el maximo
    Serial.println( "El maximo de la matriz es " + String(maximo) );
}
```

5. Ejercicio.

1. Hacer un programa que lea un vector de enteros de tamaño *TAM* (usar la función de este tema) y devuelva la media de los elementos con valor par mediante la función **media_de_pares(V, tam)**. Por ejemplo, si el vector contiene *[1,2,3,1,4,6,7,8,0,9]* el resultado sería $(2+4+6+8+0) / 5 = 4.0$.
2. Hacer un programa que lea un vector de tamaño TAM de *float* (usar la función de este tema) y devuelva en un vector de enteros pasado como parámetro los valores del vector redondeados a entero mediante la función **redondea_vector(F, V, tam)**. Por ejemplo, si el vector contiene *[4.3, 6.7, 1.5, 2.1]* devolvería el vector entero *[4, 7, 2, 2]*.
3. Hacer una función **suma_elementos_matriz(M, n_fil, n_col)** que devuelve la media de todos los elementos de la matriz M siendo n_fil y n_col indican el número de elementos en las filas y columnas de la matriz.
4. Implementa la función **media_elementos_columna(M, n_fil, n_col, col)** que devuelve la media de los elementos de la columna col/ de la matriz M.

6. Conclusiones

1. En **C estandar** no hay nada equivalente a las listas y tuplas de Python. Son un elemento con muchas más posibilidades que los vectores de C.
2. Los **vectores** son una colección de datos del mismo tipo que se referencian con un nombre común y utilizando un índice.
3. La forma de **recorrer los vectores** es mediante el uso de bucles. Normalmente, un *for* será lo más adecuado.
4. Las matrices **son una colección de datos del mismo tipo con dos o más dimensiones** que se referencian con un nombre común y utilizando un índice.
5. En ambos casos **se deben hacer funciones sencillas**, si son más complicadas se deben **invocar a otras funciones dentro de las éstas**.